

Scenario - Using Microservices and Event-Driven Architecture in an Online shopping platform.

An online shopping app shopEase.

→ How Microservices Architecture used.

- Breaking down the system: shopEase divides its platform into small, independent services such as
 - Product service (handles product details)
 - Order service (manages orders)
 - Payment service (process payments)
 - Inventory service (tracks stock)
 - User service (manages user accounts)

Each service works separately.

→ How Event driven Architecture used.

- Event managing system to let services communicate by sending and receiving events.
- When a customer places an order, the order service sends an 'order placed' event.
- The inventory service listens for this event to reduce stock.
- The payment listens to start payment processing.

→ Using solid principles to improve design.

- ⊗ Single Responsibility: Each service does one job only, making it easier to manage & update.
- ⊗ Open/Closed: New payment service can be extended with new features.
- ⊗ Liskov Substitution: Interfaces like IPaymentGateway ensure new payment types can replace existing ones.
- ⊗ Interface Segregation: Services implement only relevant interfaces.
- ⊗ Dependency Inversion: High level services depend on abstractions.

→ DRY and KISS principles.

- DRY (Don't Repeat Yourself)
 - common code like logging or error handling is shared across services.
 - Event formats are standardized and needed to keep things consistent.
- KISS (Keep It Simple, Stupid):
 - Each service is designed to do a simple, clean task.
 - Using events instead of complex direct calls keeps communication.
 - APIs and event messages are kept minimal.